UADE – PROGRAMACIÓN III | CF FAIN N° 552 | FULL-STACK JAVA / WEB

Simulador y Motor Algorítmico de Batalla: Fate/stay night Universe

Defensa técnica integradora: Optimización combinatoria, algoritmos sobre grafos y toma de decisiones adversarial en la Guerra del Santo Grial.



01. MARCO TEÓRICO INTEGRADO

Matriz de Unidades Curriculares y Complejidad



Unidad Curricular	Estrategia Algorítmica	Aplicación en Fate/stay night	Complejidad Requerida
Unidad I	Análisis Asintótico	Cálculo formal de complejidad temporal y espacial en modelos	O(1) a O(2^n)
Unidad II	Algoritmos en Grafos	Rutas mínimas en Fuyuki (Dijkstra) y Red de Mana (Kruskal/Prim)	O((V+E) log V) / O(E log E)
Unidad III	Divide & Conquer / Greedy	Ordenamiento de Servants (MergeSort) y Selección Golosa	O(N log N)
Unidad IV	Programación Dinámica	Optimización de Fantasmas Nobles bajo límite de Mana (Knapsack 0/1)	O(N × W) Pseudo-polinomial
Unidad V	Backtracking	Búsqueda de alianzas viables entre Masters y Servants con poda moral	O(N!) Factorial con poda
Unidad VI	Minimax y Alfa-Beta	Simulador de combate adversarial por turnos entre clases de Servants	O(b^(m/2)) Con poda eficiente

02. UNIDAD II — ALGORITMOS EN GRAFOS

Modelado Geográfico y Red de Mana en Fuyuki

Dijkstra (Caminos Mínimos)

Calcula el trayecto más rápido desde la Residencia Emiya hacia la anomalía mágica evitando la degradación de Mana.

Estructura: Cola de Prioridad (PriorityQueue)
Complejidad Temporal: O((V + E) log V)

Kruskal / Prim (Árbol Mínimo)

Interconecta los nodos territoriales de líneas místicas (Leylines) minimizando el consumo total para sostener la barrera.

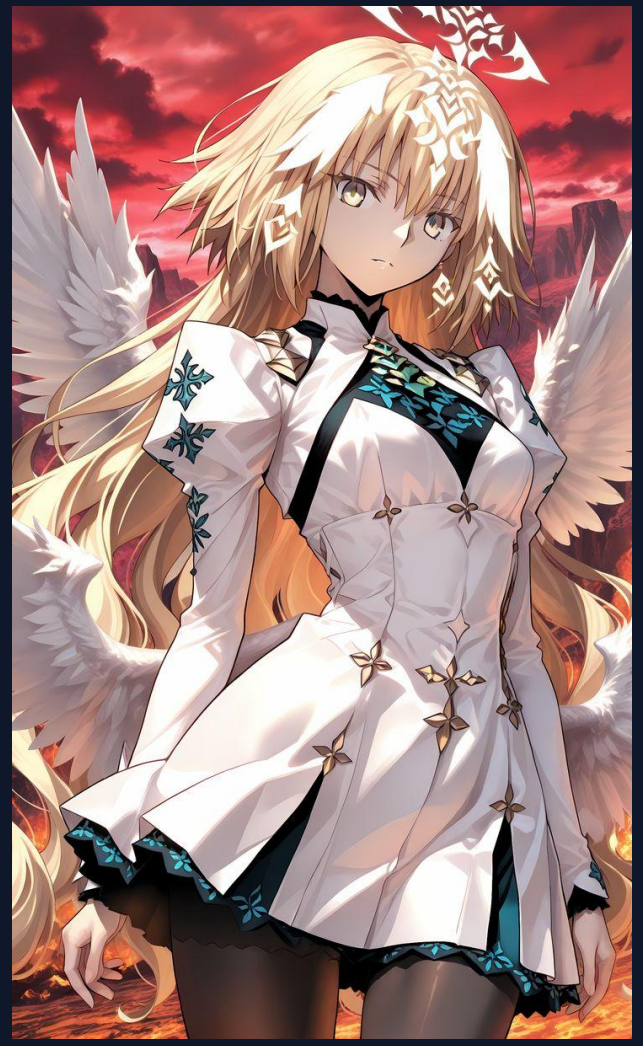
Estructura: Conjuntos Disjuntos (Union-Find)
Complejidad Temporal: O(E log E)



Algoritmo de Dijkstra en Java (`Algoritmos.java`)

```
public static Map dijkstra(Node startNode, List graph) { Map distances = new HashMap<>(); PriorityQueue pq = new PriorityQueue<>(Comparator.comparingInt(n -> n.cost)); for (Node node : graph) distances.put(node.name, Integer.MAX_VALUE); distances.put(startNode.name, 0); pq.add(new NodeCost(startNode, 0)); while (!pq.isEmpty()) { NodeCost current = pq.poll(); if (current.cost > distances.get(current.node.name)) continue; for (Edge edge : current.node.edges) { int newDist = distances.get(current.node.name) + edge.weight; if (newDist < distances.get(edge.target.name)) { distances.put(edge.target.name, newDist); pq.add(new NodeCost(edge.target, newDist)); } } } return distances; }
```

```
public static Map dijkstra(Node startNode, List graph) { Map distances = new HashMap<>(); PriorityQueue pq = new PriorityQueue<>(Comparator.comparingInt(n -> n.cost)); for (Node node : graph) distances.put(node.name, Integer.MAX_VALUE); distances.put(startNode.name, 0); pq.add(new NodeCost(startNode, 0)); while (!pq.isEmpty()) { NodeCost current = pq.poll(); if (current.cost > distances.get(current.node.name)) continue; for (Edge edge : current.node.edges) { int newDist = distances.get(current.node.name) + edge.weight; if (newDist < distances.get(edge.target.name)) { distances.put(edge.target.name, newDist); pq.add(new NodeCost(edge.target, newDist)); } } } return distances; }
```



04. UNIDADES III Y IV — OPTIMIZACIÓN COMBINATORIA

Optimización de Fantasmas Nobles y Clasificación

Greedy vs. Programación Dinámica

El enfoque Greedy por densidad daño/mana falla al no garantizar el óptimo global debido a la indivisibilidad de las habilidades.

Principio de Optimalidad de Bellman: Garantiza la selección exacta en O(N × W).

Clasificación por MergeSort

Ordenamiento estable por Divide & Conquer para jerarquizar Servants según sus atributos combinados (Fuerza, Agilidad, Mana, Suerte).

Complejidad Determinista: O(N log N) espacio y tiempo.



05. EVALUACIÓN COMPARATIVA

Inventario de Fantasmas Nobles y Formulación DP

```
DP[i][w] = DP[i-1][w] if weight[i] > w else max(DP[i-1][w], DP[i-1][w - weight[i]] + value[i])
```

Noble Phantasm	Servant	Costo Mana (w_i)	Daño Base (v_i)	Ratio Eficiencia (v_i / w_i)
Excalibur	Saber (Artoria)	80 MP	950 DMG	11.87
Gáe Bolg	Lancer (Cú Chulainn)	30 MP	450 DMG	15.00
Unlimited Blade Works	Archer (EMIYA)	60 MP	700 DMG	11.66
Gate of Babylon	Gilgamesh	50 MP	850 DMG	17.00





Asignación de Alianzas Master-Servant con Backtracking

Espacio de Búsqueda Factorial

- Modelado como Búsqueda en Profundidad (DFS) en un árbol factorial O(N!) para 7 Masters y 7 Servants.
- Resuelve el problema de asignación evaluando restricciones combinadas.

Criterios Estrictos de Poda

- **Alineamiento Moral:** Compatibilidad Lawful / Neutral / Chaos.
- **Afinidad Elemental:** Sinergia de atributos mágicos.
- **Presupuesto de Mana:** Capacidad de sostén del Master.



07. UNIDAD VI — TOMA DE DECISIONES ADVERSARIAL

Motor de Combate Táctico con Poda Alfa-Beta

Evaluación de Estado

Cálculo iterativo sobre HP, Mana, Sellos de Comando y estados defensivos pasivos.

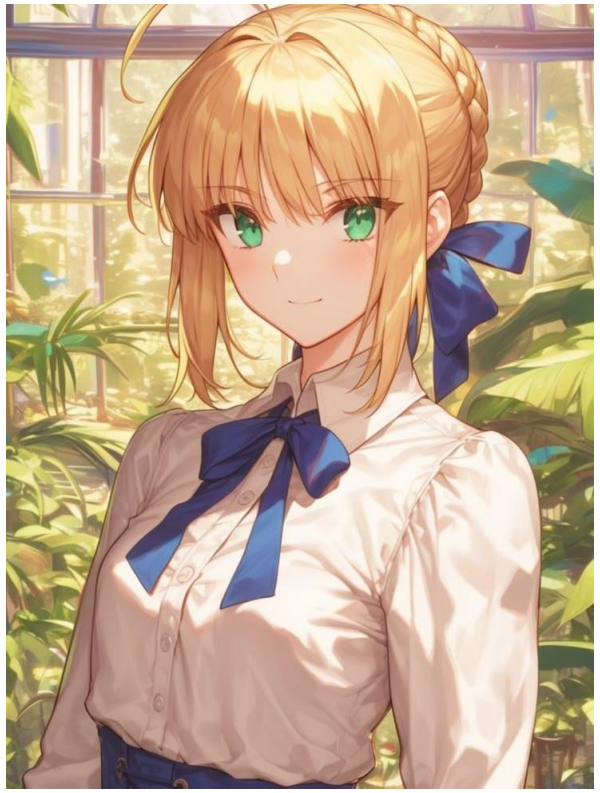
Reducción del Espacio

Reducción de complejidad de O(b^m) a O(b^(m/2)) con profundidad mínima d=4.

Rendimiento

Permite simular hasta 12 turnos por segundo cancelando ramas inútiles.



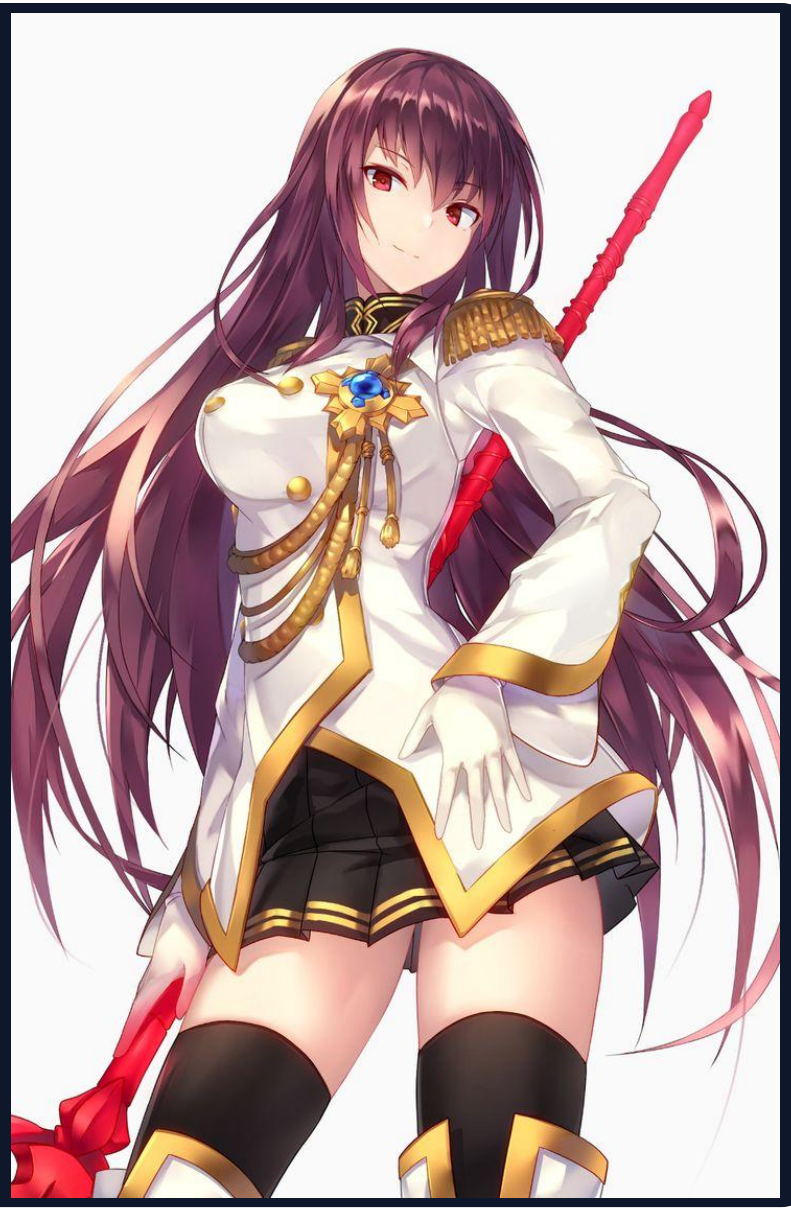




08. CÓDIGO DEL MOTOR

Algoritmo Minimax con Poda Alfa-Beta (`WebServer.java`)

```
public static int minimax(BattleState state, int depth, int alpha, int beta, boolean isMax) { if (depth == 0 || state.isGameOver()) return state.evaluateUtility(); if (isMax) { // Turno Jugador int maxEval = Integer.MIN_VALUE; for (BattleState child : state.generateSuccessors()) { int eval = minimax(child, depth - 1, alpha, beta, false); maxEval = Math.max(maxEval, eval); alpha = Math.max(alpha, eval); if (beta <= alpha) break; // Poda Beta } return maxEval; } else { // Turno Oponente int minEval = Integer.MAX_VALUE; for (BattleState child : state.generateSuccessors()) { int eval = minimax(child, depth - 1, alpha, beta, true); minEval = Math.min(minEval, eval); beta = Math.min(beta, eval); if (beta <= alpha) break; // Poda Alfa } return minEval; } }
```



09. REGLAS DE CLASE

Ventajas Múltiples y Habilidades Pasivas

Clase de Servant	Ventaja (×1.5)	Desventaja (×0.7)	Habilidad Especial Pasiva
Saber	Lancer	Archer	**Magic Resistance:** Reducción del 15% de daño de magias.
Lancer	Archer	Saber	**Battle Continuation:** Revive 1 vez con 10% HP al caer.
Archer	Saber	Lancer	**Independent Action:** Permite atacar 2 turnos sin gasto de Mana.
Caster	Berserker	Rider	**Territory Creation:** Regenera +10 MP al final de cada turno.





10. ARQUITECTURA DE SOFTWARE

Diseño Full-Stack Java SE & Frontend Web

Backend Java Nativo (Sin Frameworks)

- Java SE 17+ con servidor com.sun.net.httpserver.HttpServer.
- Endpoints REST nativos: /api/dijkstra, /api/knapsack, /api/minimax.
- Manejo estricto de errores con respuestas JSON (200, 400, 500).

Frontend Web SPA Interactivo

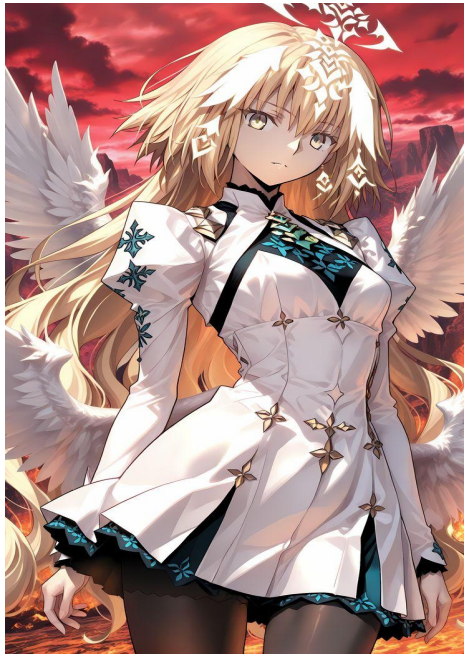
- Interfaz limpia construida en HTML5, CSS3 nativo y JavaScript (ES6).
- Comunicación asíncrona mediante API fetch().
- Renderizado en tiempo real de tableros de combate y grafos de Fuyuki.



11. ESTRUCTURA DE CÓDIGO

Catálogo de Clases Principales del Proyecto

Archivo / Clase	Capa	Responsabilidad Principal
Main.java	Punto de Entrada	Inicializa estructuras de datos globales y lanza el servidor backend.
WebServer.java	Servidor HTTP	Gestiona endpoints REST (/api/dijkstra, /api/knapsack, /api/minimax).
Algoritmos.java	Lógica Core	Implementación pura de Dijkstra, Kruskal, MergeSort, Knapsack y Minimax.
Datos.java	Modelo de Datos	Entidades Servants (Saber, Archer, Lancer), Masters y Grafo de Fuyuki.
app.js / index.html	Frontend Web	Interfaz SPA interactiva con tablero visual de batalla y mapas.





12. EVALUACIÓN Y ENTREGABLES

Ponderación de Calificación y Criterios

50%

Código Funcional

Backend Java SE y Frontend SPA sin advertencias ni fallos en vivo.

30%

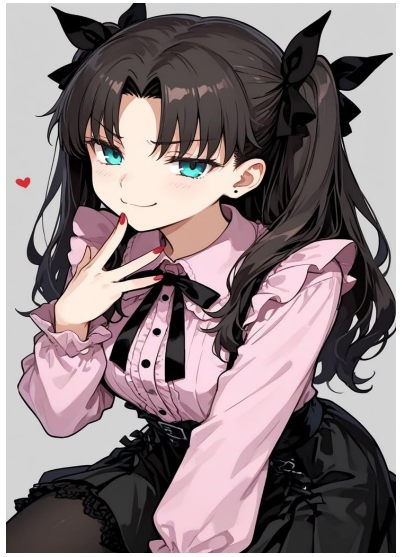
Informe Técnico PDF

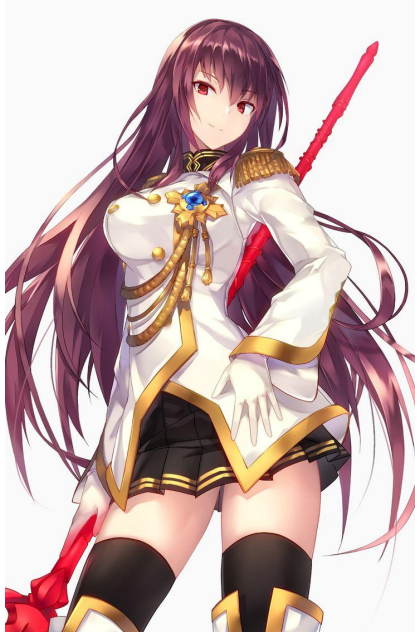
Diagramas UML, demostraciones asintóticas y justificación algorítmica.

20%

Defensa Oral

Demostración individual de dominio del código y resolución de preguntas.







Condición Obligatoria: Se requiere obtener una nota mínima de 6 (seis) puntos tanto en el desarrollo del proyecto como en la defensa oral individual.





CUMPLIMIENTO DE REQUERIMIENTOS

Conclusiones del Proyecto Integrador

El sistema valida satisfactoriamente la superioridad de la Programación Dinámica frente a enfoques golosos, la eficiencia del modelado con Dijkstra y Kruskal en grafos reales, y la solidez de Minimax con Poda Alfa-Beta para la toma de decisiones tácticas adversariales en tiempo real.

APROBACIÓN CF FAIN N° 552

UADE - PROGRAMACIÓN III

